

2D Curvilinear Grid Generation

User Manual and Algorithm Reference

July 5, 2026

Abstract

This document provides a comprehensive reference for the 2D curvilinear grid generation Python package. The package implements three grid generation methods — parabolic, hyperbolic, and elliptic — for constructing body-fitted meshes suitable for seismic wave numerical simulation. This manual covers the mathematical foundations, algorithmic details of each method, grid quality metrics, post-processing capabilities, and configuration guide.

Contents

1	Introduction	2
1.1	Background	2
1.2	Scope	2
1.3	Computational Coordinates Convention	2
2	Mathematical Foundations	4
2.1	Curvilinear Coordinate Transformation	4
2.2	Metric Tensors	4
2.3	Jacobian Determinant	4
2.4	Finite Difference Approximations	4
3	Parabolic Grid Generation	5
3.1	Governing Equation	5
3.2	Predictor Step: Algebraic Prediction	5
3.2.1	Normal Vector Computation	5
3.2.2	Prediction Points	5
3.2.3	Improved Step Coefficients	6
3.3	Corrector Step: Elliptic Solve	6
3.3.1	Discretization	6
3.3.2	Boundary Conditions	6
3.4	Thomas Algorithm	7
3.5	Marching Direction	7
4	Hyperbolic Grid Generation	8
4.1	Governing Equations	8
4.2	Linearization and Matrix Form	8
4.3	Block Tridiagonal System	9
4.4	Artificial Dissipation	9
4.5	Adaptive Dissipation Coefficient	9
4.5.1	Aspect Ratio Factor	9
4.5.2	Depth Factor	9
4.5.3	Clustering Detection Factor	9
4.5.4	Angle Factor	10
4.6	Floating Boundary Conditions	10
4.7	Arc-Length Stretching	10
4.8	Area Specification	10
5	Elliptic Grid Generation	11
5.1	Governing Equation	11
5.2	Initialization: Transfinite Interpolation (TFI)	11
5.3	SOR Iteration	11
5.3.1	Update Formula	12

5.3.2	Convergence Check	12
5.4	Source Term Computation	12
5.4.1	Dirichlet Method	12
5.4.2	Higenstock Method	12
5.5	Interior Source Interpolation	13
5.6	MPI Domain Decomposition	13
6	Grid Quality Metrics	14
6.1	Orthogonality	14
6.2	Jacobian Determinant	14
6.3	Aspect Ratio	14
6.4	Step Sizes	14
6.5	Smoothness	15
6.6	Minimum Distance (CFL Stability)	15
7	Post-Processing	16
7.1	Grid Import and Merging	16
7.1.1	Import	16
7.1.2	Merging	16
7.2	Grid Sampling (Upsampling)	16
7.3	Arc-Length Stretching	16
7.4	PML Layer Support	16
7.5	x/z Axis Swap (Reflection)	17
7.6	Automatic Output Configuration	17
8	Configuration Guide	18
8.1	JSON Format	18
8.2	Common Parameters	18
8.3	Parabolic Parameters	18
8.4	Hyperbolic Parameters	18
8.5	Elliptic Parameters	19
8.5.1	Method Sub-configuration	19
8.6	Post-Processing Parameters	19
9	Installation and Usage	20
9.1	Dependencies	20
9.2	Building the C++ Kernel	20
9.3	Running Grid Generation	20
9.3.1	Parabolic	20
9.3.2	Hyperbolic	21
9.3.3	Elliptic (MPI)	21
9.3.4	Post-Processing	21
A	File Formats	22
A.1	Geometry File	22
A.2	Step File	22
A.3	NetCDF Output Format	22
A.4	Arc-Length File	23
	References	24

Chapter 1

Introduction

1.1 Background

Curvilinear grid generation is essential for finite-difference seismic wave simulation in domains with complex topography. Unlike Cartesian grids, curvilinear grids conform to irregular boundaries, eliminating staircase approximation errors at the free surface.

This package provides Python implementations of three grid generation methods, documented in a doctoral thesis [1]:

- **Parabolic method** — A predictor-corrector scheme that marches from one boundary to another, generating grids with controlled clustering near the surface. Refer to Chapter 3 of [1] for the full mathematical derivation.
- **Hyperbolic method** — A marching method based on orthogonality and area conservation, suitable for extreme topography (steep cliffs, vertical dams). Refer to Chapter 4 of [1] for details.
- **Elliptic method** — An iterative method based on solving elliptic PDEs with source terms, supporting MPI parallel execution. This method is code-documented only (not covered in [1]).

1.2 Scope

This manual covers:

- Mathematical foundations of curvilinear coordinates
- Algorithmic principles and implementation details for all three methods
- Grid quality metrics
- Post-processing (merging, sampling, stretching, PML)
- Configuration and usage guide

1.3 Computational Coordinates Convention

Throughout this document, we use the following notation:

- Physical coordinates: (x, z)
- Computational coordinates: (ξ, ζ) , where ξ corresponds to the horizontal (x) direction and ζ corresponds to the vertical (z) direction

- In the code, ξ is referred to as **xi** and ζ as **zt**
- Grid indices: $i \in [0, N_x - 1]$ for ξ , $k \in [0, N_z - 1]$ for ζ

Chapter 2

Mathematical Foundations

2.1 Curvilinear Coordinate Transformation

The mapping from computational domain to physical domain is:

$$x = x(\xi, \zeta), \quad z = z(\xi, \zeta). \quad (2.1)$$

The position vector at each grid point is:

$$\mathbf{r} = [x, z]^T. \quad (2.2)$$

2.2 Metric Tensors

The covariant metric tensor components are defined as:

$$\begin{aligned} g_{11} &= \mathbf{r}_{,\xi} \cdot \mathbf{r}_{,\xi} = x_{,\xi}^2 + z_{,\xi}^2, \\ g_{12} &= \mathbf{r}_{,\xi} \cdot \mathbf{r}_{,\zeta} = x_{,\xi}x_{,\zeta} + z_{,\xi}z_{,\zeta}, \\ g_{22} &= \mathbf{r}_{,\zeta} \cdot \mathbf{r}_{,\zeta} = x_{,\zeta}^2 + z_{,\zeta}^2. \end{aligned} \quad (2.3)$$

2.3 Jacobian Determinant

The Jacobian determinant represents the area scaling factor of the coordinate transformation:

$$J = \begin{vmatrix} x_{,\xi} & x_{,\zeta} \\ z_{,\xi} & z_{,\zeta} \end{vmatrix} = x_{,\xi}z_{,\zeta} - z_{,\xi}x_{,\zeta}. \quad (2.4)$$

For a valid grid, $J > 0$ must hold everywhere, ensuring no cell folding or inversion.

2.4 Finite Difference Approximations

First derivatives (central differences):

$$\begin{aligned} x_{,\xi} &\approx \frac{1}{2}(x_{i+1,k} - x_{i-1,k}), \\ x_{,\zeta} &\approx \frac{1}{2}(x_{i,k+1} - x_{i,k-1}). \end{aligned} \quad (2.5)$$

Second derivatives:

$$\begin{aligned} x_{,\xi\xi} &\approx x_{i-1,k} - 2x_{i,k} + x_{i+1,k}, \\ x_{,\zeta\zeta} &\approx x_{i,k-1} - 2x_{i,k} + x_{i,k+1}. \end{aligned} \quad (2.6)$$

Mixed derivative:

$$x_{,\xi\zeta} \approx \frac{1}{4}(x_{i+1,k+1} + x_{i-1,k-1} - x_{i-1,k+1} - x_{i+1,k-1}). \quad (2.7)$$

Chapter 3

Parabolic Grid Generation

The parabolic method generates grids by marching from one boundary toward the other, using a predictor-corrector approach. It combines algebraic prediction with elliptic correction at each marching step.

3.1 Governing Equation

The parabolic grid generation equation is a quasi-elliptic PDE:

$$g_{22}f_{,\xi\xi} + g_{11}f_{,\zeta\zeta} - 2g_{12}f_{,\xi\zeta} = 0, \quad (3.1)$$

where f represents either the x or z coordinate, and the metric coefficients g_{11} , g_{12} , g_{22} are computed from the current grid.

3.2 Predictor Step: Algebraic Prediction

The predictor step generates an initial guess for the next grid layer using a combination of orthogonal projection and linear extrapolation.

3.2.1 Normal Vector Computation

Given the current layer $\mathbf{r}_{i,j}$ and the previous layer $\mathbf{r}_{i,j-1}$, define the displacement vector:

$$\mathbf{R} = \mathbf{r}_{i,j-1} - \mathbf{r}_{i,J}, \quad (3.2)$$

where J is the target boundary index.

The unit normal vector to the current grid line is:

$$\mathbf{n} = \frac{z_{,\xi}\hat{\mathbf{i}} - x_{,\xi}\hat{\mathbf{j}}}{\sqrt{x_{,\xi}^2 + z_{,\xi}^2}}. \quad (3.3)$$

3.2.2 Prediction Points

Two candidate prediction points are computed:

Orthogonal prediction (normal to current grid line):

$$\mathbf{r}_o = \mathbf{r}_{i,j-1} + \varepsilon_c |\mathbf{R}| \mathbf{n}. \quad (3.4)$$

Linear prediction (along displacement direction):

$$\mathbf{r}_s = \mathbf{r}_{i,j-1} + \varepsilon_c \mathbf{R}. \quad (3.5)$$

Final prediction (weighted combination):

$$\mathbf{r}_{i,j+1} = \varepsilon_s \mathbf{r}_o + (1 - \varepsilon_s) \mathbf{r}_s, \quad (3.6)$$

where the switching factor is:

$$\varepsilon_s = e^{-a\zeta}, \quad \zeta = \frac{j-1}{J-1}. \quad (3.7)$$

Here, a is the clustering coefficient (parameter `coef` in the configuration). A smaller a causes ε_s to decay more slowly, resulting in better grid orthogonality near the free surface, but potentially reducing grid smoothness and even causing grid crossing.

3.2.3 Improved Step Coefficients

The coefficients ε_c and ε_i are computed using normalized arc-length distances:

$$\begin{aligned} \varepsilon_c &= \frac{d_{j+1} - d_{j-1}}{d_J - d_{j-1}}, \\ \varepsilon_i &= \frac{d_j - d_{j-1}}{d_{j+1} - d_{j-1}}, \end{aligned} \quad (3.8)$$

where the cumulative arc length is:

$$d_1 = 0, \quad d_j = d_{j-1} + h_{j-1} \quad (j = 2, 3, \dots, J), \quad (3.9)$$

and h_j is the step length at layer j from the step file.

3.3 Corrector Step: Elliptic Solve

After prediction, the corrector step solves the discretized elliptic equation (3.1) using the Thomas algorithm for the resulting tridiagonal system.

3.3.1 Discretization

Discretizing Eq. (3.1) at each interior point (i, j) :

$$g_{22}(f_{i-1,j} - 2f_{i,j} + f_{i+1,j}) + g_{11}(f_{i,j-1} - 2f_{i,j} + f_{i,j+1}) - 2g_{12}f_{,\xi\zeta} = 0. \quad (3.10)$$

For each layer j , the known values are $f_{i,j-1}$ (previous layer) and $f_{i,j+1}$ (predicted layer), leading to a tridiagonal system in the ξ -direction:

$$a_i f_{i-1,j} + b_i f_{i,j} + c_i f_{i+1,j} = D_i, \quad (3.11)$$

where:

$$\begin{aligned} a_i &= g_{22}, \\ b_i &= -2(g_{11} + g_{22}), \\ c_i &= g_{22}, \\ D_i &= -g_{11}(f_{i,j-1} + f_{i,j+1}) + 2g_{12}f_{,\xi\zeta}. \end{aligned} \quad (3.12)$$

3.3.2 Boundary Conditions

At the lateral boundaries ($i = 1$ and $i = I - 1$), geometric symmetry is applied:

$$\begin{aligned} \mathbf{r}_{1,j} &= 2\mathbf{r}_{2,j} - \mathbf{r}_{3,j}, \\ \mathbf{r}_{I,j} &= 2\mathbf{r}_{I-1,j} - \mathbf{r}_{I-2,j}. \end{aligned} \quad (3.13)$$

3.4 Thomas Algorithm

The tridiagonal system is solved efficiently using the Thomas algorithm (TDMA), which runs in $O(N)$ time:

1. **Forward elimination:** For $i = 2, 3, \dots, N - 1$:

$$\begin{aligned}c'_i &= c_i / (b_i - a_i c'_{i-1}), \\ D'_i &= (D_i - a_i D'_{i-1}) / (b_i - a_i c'_{i-1}),\end{aligned}\tag{3.14}$$

with $c'_1 = c_1/b_1$ and $D'_1 = D_1/b_1$.

2. **Back substitution:** For $i = N - 1, N - 2, \dots, 1$:

$$f_i = D'_i - c'_i f_{i+1}.\tag{3.15}$$

3.5 Marching Direction

The parameter `t2b` controls the marching direction:

- `t2b = 1`: March from top boundary (boundary 2) to bottom boundary (boundary 1)
- `t2b = 0`: March from bottom boundary (boundary 1) to top boundary (boundary 2)

Chapter 4

Hyperbolic Grid Generation

The hyperbolic method generates grids by marching from a single boundary, enforcing orthogonality and area conservation at each step. It is particularly suitable for extreme topography where the parabolic method may fail.

4.1 Governing Equations

The hyperbolic grid generation is based on two conditions:

Orthogonality condition:

$$\mathbf{r}_{j,\xi} \cdot \mathbf{r}_{j,\zeta} = 0. \quad (4.1)$$

Area conservation condition:

$$\mathbf{r}_{j,\xi} \times \mathbf{r}_{j,\zeta} = \Delta A_j, \quad (4.2)$$

where ΔA_j is the specified cell area at layer j .

4.2 Linearization and Matrix Form

Linearizing Eqs. (4.1)–(4.2) about the known layer $j - 1$:

$$\mathbf{A}_{j-1} \mathbf{r}_{j,\xi} + \mathbf{B}_{j-1} \mathbf{r}_{j,\zeta} = \mathbf{f}_j, \quad (4.3)$$

where the coefficient matrices are:

$$\mathbf{A} = \begin{bmatrix} x_{j-1,\zeta} & z_{j-1,\zeta} \\ z_{j-1,\zeta} & -x_{j-1,\zeta} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} x_{j-1,\xi} & z_{j-1,\xi} \\ -z_{j-1,\xi} & x_{j-1,\xi} \end{bmatrix}, \quad (4.4)$$

$$\mathbf{f} = \begin{bmatrix} 0 \\ \Delta A_{j-1} + \Delta A_j \end{bmatrix}. \quad (4.5)$$

The ζ -direction derivatives are approximated from the known layer:

$$\begin{aligned} x_{j-1,\zeta} &= -\frac{z_{j-1,\xi} \Delta A_{j-1}}{x_{j-1,\xi}^2 + z_{j-1,\xi}^2}, \\ z_{j-1,\zeta} &= \frac{x_{j-1,\xi} \Delta A_{j-1}}{x_{j-1,\xi}^2 + z_{j-1,\xi}^2}. \end{aligned} \quad (4.6)$$

4.3 Block Tridiagonal System

Writing the system in terms of the increment $\Delta \mathbf{r}_j = \mathbf{r}_j - \mathbf{r}_{j-1}$ and applying central differences for ξ -derivatives yields a block tridiagonal system with 2×2 blocks at each grid point:

$$(\mathbf{I} + \mathbf{B}_{j-1}^{-1} \mathbf{A}_{j-1} D_\xi) \Delta \mathbf{r}_j = \mathbf{B}_{j-1}^{-1} \mathbf{e}_j, \quad (4.7)$$

where $\mathbf{e}_j = [0, \Delta A_j]^T$ and D_ξ is the central difference operator.

This system is solved using the block Thomas algorithm (ThomasBlock), which generalizes the scalar Thomas algorithm to 2×2 block matrices.

4.4 Artificial Dissipation

To ensure stability and smoothness, artificial dissipation terms are added:

$$(\mathbf{I} + \mathbf{B}_{j-1}^{-1} \mathbf{A}_{j-1} D_\xi - \varepsilon_{i\xi} (\Delta \nabla)_\xi) \Delta \mathbf{r}_j = \mathbf{B}_{j-1}^{-1} \mathbf{e}_j + \varepsilon_{e\xi} (\Delta \nabla)_\xi \mathbf{r}_{j-1}, \quad (4.8)$$

where the second-order dissipation operator is:

$$(\Delta \nabla)_\xi \mathbf{r} = \mathbf{r}_{i-1} + \mathbf{r}_{i+1} - 2\mathbf{r}_i, \quad (4.9)$$

and $\varepsilon_{i\xi} = 2\varepsilon_{e\xi}$.

4.5 Adaptive Dissipation Coefficient

The dissipation coefficient adapts to local grid geometry:

$$\varepsilon_{e\xi} = \varepsilon_c \cdot N_\xi \cdot S \cdot \hat{d}_\xi \cdot a_\xi, \quad (4.10)$$

where ε_c is the base coefficient (parameter `coef` in config), and the four factors are:

4.5.1 Aspect Ratio Factor

$$N_\xi = \sqrt{\frac{x_{,\zeta}^2 + z_{,\zeta}^2}{x_{,\xi}^2 + z_{,\xi}^2}}. \quad (4.11)$$

4.5.2 Depth Factor

$$S = \sqrt{\frac{j-1}{J-1}}. \quad (4.12)$$

4.5.3 Clustering Detection Factor

$$\hat{d}_\xi = \max[d_\xi^{2/S}, 0.1], \quad (4.13)$$

$$d_\xi = \frac{|\mathbf{r}_{i+1,j-2} - \mathbf{r}_{i,j-2}| + |\mathbf{r}_{i,j-2} - \mathbf{r}_{i-1,j-2}|}{|\mathbf{r}_{i+1,j-1} - \mathbf{r}_{i,j-1}| + |\mathbf{r}_{i,j-1} - \mathbf{r}_{i-1,j-1}|}. \quad (4.14)$$

4.5.4 Angle Factor

The angle factor detects concave corners where additional dissipation is needed. Define unit direction vectors:

$$\begin{aligned} \mathbf{r}_\xi^+ &= \frac{\mathbf{r}_{i+1,j-1} - \mathbf{r}_{i,j-1}}{|\mathbf{r}_{i+1,j-1} - \mathbf{r}_{i,j-1}|}, \\ \mathbf{r}_\xi^- &= \frac{\mathbf{r}_{i-1,j-1} - \mathbf{r}_{i,j-1}}{|\mathbf{r}_{i-1,j-1} - \mathbf{r}_{i,j-1}|}. \end{aligned} \quad (4.15)$$

Compute the angle θ at the grid point:

$$\beta = \text{atan2}(-(\mathbf{r}_\xi^+ \times \mathbf{r}_\xi^-), (\mathbf{r}_\xi^+ \cdot \mathbf{r}_\xi^-)), \quad (4.16)$$

$$\theta = \begin{cases} \beta + 2\pi & \text{if } \beta < 0, \\ \beta & \text{if } \beta \geq 0. \end{cases} \quad (4.17)$$

The angle factor is:

$$a_\xi = \begin{cases} \frac{1}{1 - \cos^2(\theta/2)} & \text{if } 0 < \theta < \pi, \\ 1 & \text{if } \pi \leq \theta \leq 2\pi. \end{cases} \quad (4.18)$$

4.6 Floating Boundary Conditions

At the lateral boundaries, floating (Neumann-like) conditions are applied:

$$\begin{aligned} \Delta \mathbf{r}_{1,j} &= \Delta \mathbf{r}_{2,j}, \\ \Delta \mathbf{r}_{I,j} &= \Delta \mathbf{r}_{I-1,j}. \end{aligned} \quad (4.19)$$

4.7 Arc-Length Stretching

After grid generation, points can be redistributed along each vertical line using arc-length stretching. The cumulative arc length along a vertical grid line is:

$$s_1 = 0, \quad s_k = s_{k-1} + |\mathbf{r}_k - \mathbf{r}_{k-1}| \quad (k = 2, 3, \dots, N_z). \quad (4.20)$$

New grid points are placed at specified arc-length positions via linear interpolation.

4.8 Area Specification

The cell area at each layer is determined by the step file:

$$\Delta A_j = \Delta s_{j-1} \cdot dh_j, \quad (4.21)$$

where Δs_{j-1} is the arc length of the current grid line and dh_j is the step length from the step file.

Chapter 5

Elliptic Grid Generation

The elliptic method generates grids by iteratively solving elliptic PDEs with source terms. Unlike the marching methods, all four boundaries are fixed and the interior grid is determined by the solution of the PDE. The method supports MPI parallel execution for large-scale grids.

5.1 Governing Equation

The elliptic grid generation system with source terms P and Q is:

$$g_{22}\mathbf{r}_{,\xi\xi} - 2g_{12}\mathbf{r}_{,\xi\zeta} + g_{11}\mathbf{r}_{,\zeta\zeta} + g_{22}P\mathbf{r}_{,\xi} + g_{11}Q\mathbf{r}_{,\zeta} = 0. \quad (5.1)$$

The source terms P and Q control grid spacing and clustering. Without them ($P = Q = 0$), the equation reduces to the Laplace equation, producing smooth but potentially non-uniform grids.

5.2 Initialization: Transfinite Interpolation (TFI)

The initial grid is generated using linear transfinite interpolation from the four boundaries. Let $\xi = i/(N_x - 1)$ and $\zeta = k/(N_z - 1)$ denote normalized coordinates.

The TFI formula is:

$$\mathbf{r}(\xi, \zeta) = \mathbf{U}(\xi, \zeta) + \mathbf{W}(\xi, \zeta) - \mathbf{U}\mathbf{W}(\xi, \zeta), \quad (5.2)$$

where:

ξ -direction interpolation (from x-boundaries):

$$\mathbf{U}(\xi, \zeta) = (1 - \xi)\mathbf{B}_{x1}(\zeta) + \xi\mathbf{B}_{x2}(\zeta), \quad (5.3)$$

ζ -direction interpolation (from z-boundaries):

$$\mathbf{W}(\xi, \zeta) = (1 - \zeta)\mathbf{B}_{z1}(\xi) + \zeta\mathbf{B}_{z2}(\xi), \quad (5.4)$$

Bilinear corner interpolation (to avoid double-counting):

$$\begin{aligned} \mathbf{U}\mathbf{W}(\xi, \zeta) = & (1 - \xi)(1 - \zeta)\mathbf{B}_{x1}(0) + (1 - \xi)\zeta\mathbf{B}_{x1}(N_z - 1) \\ & + \xi(1 - \zeta)\mathbf{B}_{x2}(0) + \xi\zeta\mathbf{B}_{x2}(N_z - 1). \end{aligned} \quad (5.5)$$

5.3 SOR Iteration

Equation (5.1) is solved iteratively using the successive over-relaxation (SOR) method (Gauss-Seidel with relaxation factor ω ; currently $\omega = 1.0$).

5.3.1 Update Formula

For each interior point (k, i) , the new value is computed as:

$$\mathbf{r}_{k,i}^{\text{tmp}} = \frac{1}{2(g_{11} + g_{22})} [g_{22}(\mathbf{r}_{k,i+1} + \mathbf{r}_{k,i-1}) + g_{11}(\mathbf{r}_{k+1,i} + \mathbf{r}_{k-1,i}) - 2g_{12}\mathbf{r}_{,\xi\zeta} + g_{22}P\mathbf{r}_{,\xi} + g_{11}Q\mathbf{r}_{,\zeta}], \quad (5.6)$$

$$\mathbf{r}_{k,i}^{\text{new}} = \omega \mathbf{r}_{k,i}^{\text{tmp}} + (1 - \omega) \mathbf{r}_{k,i}^{\text{old}}. \quad (5.7)$$

Gauss-Seidel ordering is used: updated values are immediately used for subsequent points.

5.3.2 Convergence Check

The iteration converges when the maximum normalized residual falls below `iter_err`:

$$\max_i \frac{|\Delta \mathbf{r}_i|}{h_i} < \epsilon, \quad (5.8)$$

where $\Delta \mathbf{r} = \mathbf{r}^{\text{new}} - \mathbf{r}^{\text{old}}$, h_i is the local grid spacing, and ϵ is the convergence tolerance.

The global maximum is computed via `MPI_Allreduce` with `MPI_MAX`.

5.4 Source Term Computation

Two methods are available for computing boundary source terms: Dirichlet and Higenstock.

5.4.1 Dirichlet Method

The Dirichlet method computes boundary source terms from the second derivatives of the grid coordinates.

For each boundary point, the source terms are:

$$\begin{aligned} P_{\text{bdry}} &= - \left(\frac{\mathbf{r}_{,\xi} \cdot \mathbf{r}_{,\xi\xi}}{g_{11}} + \frac{\mathbf{r}_{,\xi} \cdot \mathbf{r}_{,\zeta\zeta}}{g_{22}} \right), \\ Q_{\text{bdry}} &= - \left(\frac{\mathbf{r}_{,\zeta} \cdot \mathbf{r}_{,\xi\xi}}{g_{11}} + \frac{\mathbf{r}_{,\zeta} \cdot \mathbf{r}_{,\zeta\zeta}}{g_{22}} \right). \end{aligned} \quad (5.9)$$

Ghost point calculation: For physical boundaries, ghost points are computed by projecting the interior gradient onto the boundary normal:

1. Compute the tangent vector along the boundary.
2. Compute the outward normal: $\mathbf{n} = (z_{,\zeta}, -x_{,\zeta})/|\mathbf{r}_{,\zeta}|$.
3. Project the interior gradient: $\Delta \mathbf{r}_n = (\Delta \mathbf{r} \cdot \mathbf{n})\mathbf{n}$.
4. Set ghost point: $\mathbf{r}_{\text{ghost}} = \mathbf{r}_{\text{bdry}} - \Delta \mathbf{r}_n$.

5.4.2 Higenstock Method

The Higenstock method uses angle-based control with `tanh` feedback to enforce orthogonality at boundaries.

The angle between grid lines at the boundary is:

$$\theta = \arccos \left(\frac{\mathbf{r}_{,\xi} \cdot \mathbf{r}_{,\zeta}}{|\mathbf{r}_{,\xi}| |\mathbf{r}_{,\zeta}|} \right). \quad (5.10)$$

Source terms are updated iteratively with feedback:

$$\begin{aligned} P &\leftarrow P + a \tanh\left(\frac{dx - |\mathbf{r}_\xi|}{dx}\right), \\ Q &\leftarrow Q - a \tanh\left(\frac{\theta_0 - \theta}{\theta_0}\right), \end{aligned} \tag{5.11}$$

where $\theta_0 = \pi/2$ (target orthogonality) and a is a feedback gain parameter. The signs are adjusted for each boundary direction.

5.5 Interior Source Interpolation

Boundary source terms are interpolated to the interior using exponential decay weighting:

$$\begin{aligned} P &= w_0 [(1 - \xi)P_{x1} + \xi P_{x2}] + w_1 [e^{-c_2 \zeta} P_{z1} + e^{-c_3(1-\zeta)} P_{z2}], \\ Q &= w_0 [e^{-c_0 \xi} Q_{x1} + e^{-c_1(1-\xi)} Q_{x2}] + w_1 [(1 - \zeta)Q_{z1} + \zeta Q_{z2}], \end{aligned} \tag{5.12}$$

where $c = [c_0, c_1, c_2, c_3]$ are the **coef** parameters and $w = [w_0, w_1]$ are the **weight** parameters.

The exponential decay ensures that source terms from one boundary diminish toward the opposite boundary, preventing unintended grid distortion far from the controlling boundary.

5.6 MPI Domain Decomposition

The elliptic solver uses a 2D Cartesian topology for MPI parallelization:

1. **Domain decomposition:** The global grid is divided into $N_{px} \times N_{pz}$ subdomains, each assigned to one MPI rank.
2. **Ghost cells:** Each subdomain has one layer of ghost cells on each side, exchanged with neighboring ranks after each SOR sweep.
3. **Communication:**
 - `MPI_Sendrecv` (or `MPI_Neighbor_alltoallw`) for ghost cell exchange
 - `MPI_Allreduce(..., MPI_SUM)` for boundary source term aggregation
 - `MPI_Allreduce(..., MPI_MAX)` for convergence check
 - `MPI_Bcast` for initial configuration broadcast
4. **Load balancing:** All ranks perform the same number of SOR iterations; convergence is checked globally.

Chapter 6

Grid Quality Metrics

6.1 Orthogonality

The orthogonality metric measures the deviation from 90° between intersecting grid lines:

$$Q = 90 - |\theta - 90|, \quad (6.1)$$

where $\theta = \arccos\left(\frac{\mathbf{r}_{,\xi} \cdot \mathbf{r}_{,\zeta}}{|\mathbf{r}_{,\xi}| |\mathbf{r}_{,\zeta}|}\right)$.

Perfect orthogonality yields $Q = 90$. Values below 45 indicate poor grid quality.

6.2 Jacobian Determinant

The Jacobian represents the signed area of grid cells:

$$J = x_{,\xi} z_{,\zeta} - z_{,\xi} x_{,\zeta}. \quad (6.2)$$

Negative values indicate cell folding (grid inversion). Zero values indicate degenerate cells.

6.3 Aspect Ratio

The aspect ratio measures the ratio of adjacent cell dimensions:

$$R = \max\left(\frac{|\mathbf{r}_{,\xi}|}{|\mathbf{r}_{,\zeta}|}, \frac{|\mathbf{r}_{,\zeta}|}{|\mathbf{r}_{,\xi}|}\right). \quad (6.3)$$

$R = 1$ indicates a square cell. Large values indicate highly elongated cells.

6.4 Step Sizes

The physical step lengths in each direction:

$$\begin{aligned} \Delta s_\xi &= |\mathbf{r}_{i+1,k} - \mathbf{r}_{i,k}|, \\ \Delta s_\zeta &= |\mathbf{r}_{i,k+1} - \mathbf{r}_{i,k}|. \end{aligned} \quad (6.4)$$

The minimum step size determines the CFL stability limit for seismic wave simulation.

6.5 Smoothness

The smoothness metric measures the ratio of adjacent step lengths:

$$\begin{aligned} S_\xi &= \max \left(\frac{|\mathbf{r}_{i+1} - \mathbf{r}_i|}{|\mathbf{r}_i - \mathbf{r}_{i-1}|}, \frac{|\mathbf{r}_i - \mathbf{r}_{i-1}|}{|\mathbf{r}_{i+1} - \mathbf{r}_i|} \right), \\ S_\zeta &= \max \left(\frac{|\mathbf{r}_{k+1} - \mathbf{r}_k|}{|\mathbf{r}_k - \mathbf{r}_{k-1}|}, \frac{|\mathbf{r}_k - \mathbf{r}_{k-1}|}{|\mathbf{r}_{k+1} - \mathbf{r}_k|} \right). \end{aligned} \tag{6.5}$$

$S = 1$ indicates uniform spacing. Large values indicate abrupt changes in grid spacing.

6.6 Minimum Distance (CFL Stability)

The function `cal_min_dist` computes the minimum point-to-line distance across all interior grid cells, which determines the maximum stable time step for the finite-difference scheme via the CFL condition.

For each interior point, four point-to-line distances are computed (corresponding to the four cells surrounding the point):

$$d = \frac{|Ax_0 + Bz_0 + C|}{\sqrt{A^2 + B^2}}, \tag{6.6}$$

where A, B, C define the line through two adjacent grid points.

Chapter 7

Post-Processing

7.1 Grid Import and Merging

The post-processor reads grid coordinates from MPI-partitioned NetCDF files and optionally merges multiple grids along the x or z direction.

7.1.1 Import

Each partition is identified by its `px{i}_pz{k}` filename, containing global index and count attributes that specify the position and size of the partition within the global grid.

7.1.2 Merging

When merging along the x-direction, grids are concatenated horizontally with one overlapping column removed at each junction. Similarly for the z-direction:

$$N_x^{\text{total}} = \sum_i N_x^{(i)} - (n_{\text{grids}} - 1). \quad (7.1)$$

7.2 Grid Sampling (Upsampling)

Grid sampling increases the resolution of an existing grid using bilinear interpolation. Given a sampling factor (s_x, s_z) , the new grid dimensions are:

$$N'_x = (N_x - 1)s_x + 1, \quad N'_z = (N_z - 1)s_z + 1. \quad (7.2)$$

The interpolated coordinates at sub-grid position (α, β) within cell (i, k) are:

$$\begin{aligned} \mathbf{r}(\alpha, \beta) = & (1 - \alpha)(1 - \beta)\mathbf{r}_{i,k} + \alpha(1 - \beta)\mathbf{r}_{i+1,k} \\ & + (1 - \alpha)\beta\mathbf{r}_{i,k+1} + \alpha\beta\mathbf{r}_{i+1,k+1}. \end{aligned} \quad (7.3)$$

7.3 Arc-Length Stretching

Arc-length stretching redistributes grid points along each grid line according to a specified arc-length distribution, enabling non-uniform spacing (e.g., finer near the surface, coarser at depth).

7.4 PML Layer Support

The post-processor supports PML (Perfectly Matched Layer) partitioning, which doubles the computational weight of PML regions when distributing the grid across MPI ranks for seismic simulation.

7.5 x/z Axis Swap (Reflection)

When parabolic/hyperbolic grids are generated along the x direction (free surface on a vertical `bx1/bx2` boundary), the boundary is first reflected into the z direction via a $y = x$ mirror (`creat_bz_bdry.py`) so the marching machinery — which always marches along $\zeta(z)$ — can be reused. After generation, the post-processor swaps the grid back to physical coordinates when `flag_swap_xz = 1`:

$$(x, z)_{\text{phys}} = (z, x)_{\text{gen}}^T, \quad (7.4)$$

i.e. the x and z arrays are exchanged and the array is transposed. The swap is performed immediately after `read_import_coord`, so every subsequent step (sampling, quality checks, export) operates on the physical-coordinate grid. The swapped N_x and N_z (and the corresponding MPI partitioning, PML counts, and sampling factors) are exchanged accordingly.

7.6 Automatic Output Configuration

After processing, `post_pro.py` writes a `config.json` into `grid_export_dir` containing the post-swap grid dimensions and output MPI layout:

```
1 {
2     "number_of_grid_points_x": <post-swap Nx>,
3     "number_of_grid_points_z": <post-swap Nz>,
4     "number_of_mpiprocs_x": ...,
5     "number_of_mpiprocs_z": ...,
6     "grid_export_dir": "... "
7 }
```

Downstream tools (e.g. `plotting/draw_grid.py`) read this file directly, so the user does not need to maintain grid dimensions by hand when a swap changes $N_x \leftrightarrow N_z$.

Chapter 8

Configuration Guide

8.1 JSON Format

All parameters are specified in a JSON configuration file. Keys prefixed with `#` are treated as comments and ignored by the parser.

8.2 Common Parameters

Key	Type	Description
<code>number_of_grid_points_x</code>	int	Number of grid points in x direction
<code>number_of_grid_points_z</code>	int	Number of grid points in z direction
<code>execute_C_code</code>	0/1	Use C++ kernel acceleration (1) vs. pure Python (0)
<code>grid_check</code>	0/1	Enable grid quality checks
<code>check_orth</code>	0/1	Check orthogonality
<code>check_jac</code>	0/1	Check Jacobian
<code>check_ratio</code>	0/1	Check aspect ratio
<code>check_step_xi</code>	0/1	Check step size in xi
<code>check_step_zt</code>	0/1	Check step size in zt
<code>check_smooth_xi</code>	0/1	Check smoothness in xi
<code>check_smooth_zt</code>	0/1	Check smoothness in zt
<code>geometry_input_file</code>	string	Path to boundary geometry file
<code>grid_export_dir</code>	string	Output directory

8.3 Parabolic Parameters

Key	Type	Description
<code>step_input_file</code>	string	Step length file (nz-1 lines)
<code>coef</code>	int	Clustering coefficient a in $\varepsilon_s = e^{-a\zeta}$
<code>t2b</code>	0/1	Marching direction: 1=top→bottom

8.4 Hyperbolic Parameters

Key	Type	Description
-----	------	-------------

<code>step_input_file</code>	string	Step length file (nz-1 lines)
<code>flag_stretch</code>	0/1	Enable arc-length stretching
<code>coef</code>	int	Base dissipation coefficient ε_c
<code>t2b</code>	0/1	Marching direction: 1=top→bottom

8.5 Elliptic Parameters

Key	Type	Description
<code>number_of_mpiprocs_x</code>	int	MPI processes in x direction
<code>number_of_mpiprocs_z</code>	int	MPI processes in z direction
<code>method</code>	object	Sub-method config (§8.5.1)

8.5.1 Method Sub-configuration

Choose either `dirichlet` or `higenstock`:

Key	Type	Description
<code>coef</code>	[int×4]	Source term coefficients $[c_0, c_1, c_2, c_3]$ for 4 boundaries
<code>weight</code>	[float×2]	Interpolation weights $[w_0, w_1]$ for ξ and ζ directions
<code>iter_err</code>	float	Convergence tolerance ϵ
<code>max_iter</code>	int	Maximum SOR iterations

8.6 Post-Processing Parameters

Key	Type	Description
<code>input_grid_number</code>	int	Number of input grids
<code>number_of_mpiprocs_out</code>	[int, int]	Output MPI partitioning $[n_x, n_z]$
<code>merge_direction</code>	string	Merge direction: "x" or "z"
<code>flag_sample</code>	0/1	Enable bilinear sampling
<code>sample_factor</code>	[int, int]	Upsampling factors $[s_x, s_z]$
<code>flag_swap_xz</code>	0/1	Swap x/z axes (reflect grid back to physical coordinates)
<code>pml_weight_2x</code>	0/1	Double PML computation weight
<code>number_of_pml_x1</code>	int	PML layers at x1 boundary
<code>number_of_pml_x2</code>	int	PML layers at x2 boundary
<code>number_of_pml_z1</code>	int	PML layers at z1 boundary
<code>number_of_pml_z2</code>	int	PML layers at z2 boundary

Chapter 9

Installation and Usage

9.1 Dependencies

- Python ≥ 3.10
- NumPy
- netCDF4
- mpi4py (elliptic method only)
- Matplotlib (plotting only)
- g++ ≥ 4.9 (C++14, for building the pybind11 kernel)
- pybind11 (header-only; include path auto-detected at build time)

9.2 Building the C++ Kernel

```
1 cd src_cpp && bash build.sh
```

This compiles `gridcpp.so`, a pybind11 extension that exchanges arrays with Python through zero-copy `numpy` buffers at runtime. The build script auto-detects the Python and pybind11 include paths; no CMake is required. The hot kernels (parabolic `predict_point/update_point`, hyperbolic `cal_matrix/thomas_block`, elliptic `update_SOR/TFI`) live in this C++ layer, while Python orchestrates the marching and I/O.

9.3 Running Grid Generation

9.3.1 Parabolic

```
1 cd parabolic/generate_grid
2 bash start.sh
3 # Or directly:
4 python para.py --config-file config.json --verbose 10
```

9.3.2 Hyperbolic

```
1 cd hyperbolic/generate_grid
2 bash start.sh
3 # Or directly:
4 python hyper.py --config-file config.json --verbose 10
```

9.3.3 Elliptic (MPI)

```
1 cd elliptic/generate_grid
2 bash start.sh
3 # Or directly:
4 mpiexec -np 4 python ellip.py --config-file config.json --verbose 10
```

9.3.4 Post-Processing

```
1 cd grid_post_process
2 bash start.sh
3 # Or directly:
4 python post_pro.py --config-file config.json --verbose 10
```


Appendix A

File Formats

A.1 Geometry File

For parabolic/hyperbolic methods, the geometry file contains boundary coordinates:

- Parabolic: $2 \times N_x$ lines (two boundaries), each line: `x z`
- Hyperbolic: N_x lines (one boundary), each line: `x z`
- Elliptic: 4 boundary blocks, each preceded by a `# boundary_x` comment line
- Lines starting with `#` are comments

A.2 Step File

The step file specifies the marching step length at each layer:

- $N_z - 1$ lines, each containing a float value
- Lines starting with `#` are comments

A.3 NetCDF Output Format

Output files follow the naming convention `{prefix}_px{i}_pz{k}.nc`, where:

- `prefix`: `coord` for coordinates, or quality metric name (`orth`, `jacobi`, etc.)
- `px{i}_pz{k}`: MPI partition index

Each file contains:

- Global attributes: `global_index_of_first_physical_points` and `count_of_physical_points`
- Variables: dimensions (`k`, `i`) with `float32` type
- For coordinate files: variables `x` and `z`
- For quality files: one variable named after the quality metric

A.4 Arc-Length File

The arc-length file specifies the cumulative arc-length distribution for stretching:

- N float values (one per line), monotonically increasing
- Lines starting with `#` are comments
- C format: first line is the number of points, followed by N values

References

1. Doctoral thesis: “Seismic Wave Numerical Simulation Methods Under Complex Topography Conditions” — Chapters 3 (parabolic method) and 4 (hyperbolic method).